

Assignment 4 – ADC differansemåling med ATmega32

Oskar Hellebust-Nygaard

23. januar 2026

Innhold

1	Innledning	2
2	Kobling	2
3	Prinsipp og kravoppfyllelse	3
4	Kodeforklaring (kort)	4
5	Programkode	4
6	Resultat og observasjoner	7
7	Konklusjon	7

1 Innledning

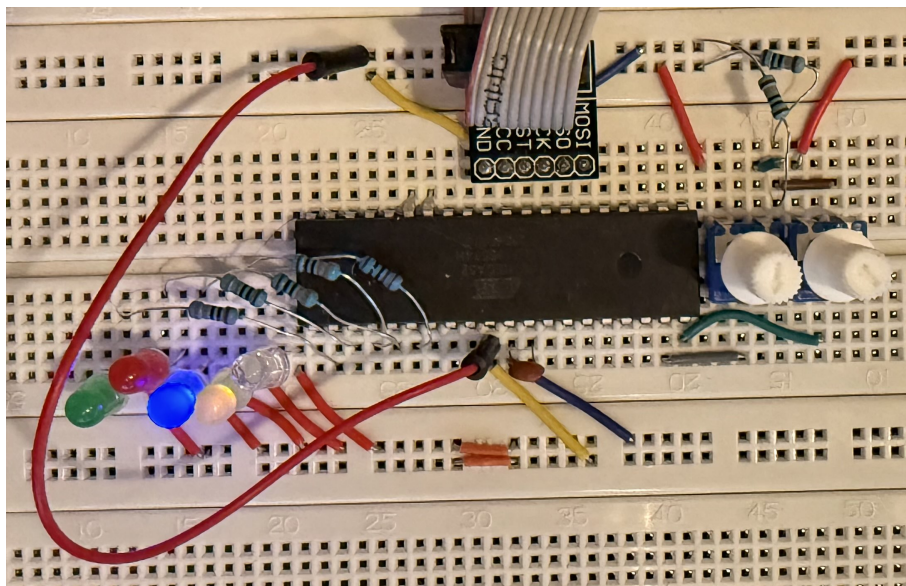
I denne oppgaven skulle vi lage et program som måler differansen mellom to analoge spenninger, $V_{in2} - V_{in1}$, med en oppløsning på 5 mV eller bedre. Basert på denne differansen skulle vi tenne en bestemt LED (og holde de andre av) innenfor gitte intervaller, og skru på alle LED-ene ellers.

Det er i tillegg krav om at vi skal bruke **ADC Noise Reduction Mode**, **ADC auto trigger** med sampling hver ca. 262 ms, at event-loopen skal være tom, og at vi skal bruke **switch** i stedet for **if**.

2 Kobling

Koblingen består av:

- Vin1 kobles til ADC0 (PA0).
- Vin2 kobles til ADC1 (PA1).
- LED0–LED4 kobles til PB0–PB4 (hver LED med seriemotstand, 330 Ω til GND).
- VCC og AVCC kobles til +5 V, GND kobles til jord.
- AREF har en 100 nF kondensator til GND.



Figur 1: Oppkobling av kretsen (Vin1/Vin2 og LED-er).

Hvorfor kondensator fra AREF til GND

Kondensatoren mellom AREF og GND fungerer som avkobling og hjelper til med å dempe støy og spenningssvingninger på referansen. Det gir mer stabile ADC-målinger, spesielt når vi er ute etter en bestemt oppløsning.

3 Prinsipp og kravoppfyllelse

Oppløsning

Med intern referanse på 2.56 V og 10-bit ADC får vi omtrent:

$$\frac{2560 \text{ mV}}{1024} \approx 2.5 \text{ mV per LSB}$$

Dette er bedre enn kravet om 5 mV.

Sampling hver 262 ms

ADC trigges automatisk av en timer (Timer1 Compare Match B) hver ca. 262 ms. Dette gjør at vi slipper polling og delay, og samtidig følger vi kravet om fast sample-rate.

Utrekning av timer-intervallet (1 MHz)

ATmega32 kjører på 1 MHz, så Timer1 må settes opp deretter.

Vi bruker prescaler 8 for Timer1:

$$f_{\text{timer}} = \frac{1\,000\,000}{8} = 125\,000 \text{ Hz}$$

Det betyr at en timer-tick er:

$$T_{\text{tick}} = \frac{1}{125\,000} = 8 \mu s$$

Vi vil ha ca. 262 ms mellom triggere:

$$N = \frac{262.144 \text{ ms}}{8 \mu s} = \frac{0.262144}{0.000008} = 32768 \text{ ticks}$$

Siden Timer1 teller fra 0 til OCR1A i CTC, blir:

$$OCR1A = 32768 - 1 = 32767$$

Vi setter også *OCR1B* = 32767 slik at Compare Match B skjer på samme tidspunkt (og brukes som ADC trigger).

Tom event-loop

Hovedløkka er tom. CPU settes i ADC Noise Reduction sleep, og vekkes av ADC-interrupt. Det gjør at vi både følger oppgaven og får mindre støy i målingene.

Switch i stedet for if

Klassifisering av differansen er gjort med `switch` (ikke `if-ladder`). LED-valg er også gjort med `switch`, slik oppgaven krever.

Clearing av timer-flag

Når ADC auto trigger bruker timer, må timer-flagget clears ved å skrive logisk 1 til flagget inne i avbruddsrutinen. Dette er lagt inn i ADC-interrupten (se linjen med `TIFR |= (1<<OCF1B);`).

4 Kodeforklaring (kort)

1. Timer1 settes i CTC slik at Compare Match B skjer med ca. 262 ms intervall.
2. ADC settes opp med:
 - intern 2.56 V referanse
 - auto trigger fra Timer1 COMPB
 - interrupt enable
 - ADC Noise Reduction Mode via sleep
3. I ADC ISR leses ADC-verdi. Vi måler annenhver gang Vin1 og Vin2, og når vi har begge beregner vi $\Delta V = V_{in2} - V_{in1}$ i mV, mapper til et intervall med `switch`, og tenner riktig LED.

5 Programkode

```
#define F_CPU 1000000UL
#include <avr/io.h>
#include <avr/interrupt.h>
#include <avr/sleep.h>
#include <stdint.h>

// LEDs on PORTB: LED0..LED4 = PB0..PB4
#define LED_PORT PORTB
#define LED_DDR DDRB

static void show_bucket(uint8_t b) {
    // slå av alle først
    LED_PORT &= ~(1<<PB0)|(1<<PB1)|(1<<PB2)|(1<<PB3)|(1<<PB4));

    switch (b) {
        case 0: LED_PORT |= (1<<PB0); break; // LED0
        case 1: LED_PORT |= (1<<PB1); break; // LED1
        case 2: LED_PORT |= (1<<PB2); break; // LED2
        case 3: LED_PORT |= (1<<PB3); break; // LED3
        case 4: LED_PORT |= (1<<PB4); break; // LED4
        default:
            // all on otherwise
            LED_PORT |= (1<<PB0)|(1<<PB1)|(1<<PB2)|(1<<PB3)|(1<<PB4);
            break;
    }
}

ISR(ADC_vect) {
    TIFR |= (1<<OCF1B);

    static uint16_t v1 = 0; // Vin1 (ADC0)
    static uint16_t v2 = 0; // Vin2 (ADC1)
```

```

static uint8_t phase = 0; // 0: nettopp lest Vin1, 1: nettopp lest Vin2

uint16_t adc = ADC; // les resultatet

switch (phase) {
    case 0:
        v1 = adc;
        // neste kanal: ADC1
        ADMUX = (1<<REFS1) | (1<<REFS0) | 1; // 2.56V ref, MUX=1
        phase = 1;
        break;

    default:
        v2 = adc;
        // tilbake til ADC0
        ADMUX = (1<<REFS1) | (1<<REFS0) | 0; // 2.56V ref, MUX=0
        phase = 0;

        // delta_mV = (v2 - v1) * 2560 / 1024 (ca. 2.5mV per count)
        {
            int16_t delta_counts = (int16_t)v2 - (int16_t)v1;
            int16_t delta_mV = (int16_t)((int32_t)delta_counts * 2560 / 1024);

            // Grov kvantisering i 500 mV steg -> q i ca. [-5..5]
            int8_t q = (int8_t)(delta_mV / 500);

            uint8_t bucket;
            switch (q) {
                case -5:
                case -4:
                    bucket = 0; // -2500..-1501 mV
                    break;

                case -3:
                case -2:
                    bucket = 1; // -1500..-501 mV
                    break;

                case -1:
                case 0:
                    bucket = 2; // -500..+499 mV
                    break;

                case 1:
                case 2:
                    bucket = 3; // +500..+1499 mV
                    break;

                case 3:
                case 4:
                case 5:

```

```

        bucket = 4; // +1500..+2500 mV
        break;

        default:
            bucket = 5; // all on otherwise
            break;
    }

    show_bucket(bucket);
}
break;
}
}

static void gpio_init() {
    LED_DDR |= (1<<PB0)|(1<<PB1)|(1<<PB2)|(1<<PB3)|(1<<PB4);
    LED_PORT &= ~((1<<PB0)|(1<<PB1)|(1<<PB2)|(1<<PB3)|(1<<PB4));
}

static void timer1_init_262ms_trigger() {
    TCCR1A = 0;
    TCCR1B = 0;

    // 1 MHz / 8 = 125 kHz timer-tick (8 us per tick)
    // 262.144 ms / 8 us = 32768 ticks -> OCR1A/B = 32767
    OCR1A = 32767;
    OCR1B = 32767;

    // CTC med OCR1A som TOP: WGM12=1
    // Prescaler 8: CS11=1
    TCCR1B = (1<<WGM12) | (1<<CS11);
}

static void adc_init() {
    // Intern 2.56V referanse, start på ADC0
    ADMUX = (1<<REFS1) | (1<<REFS0) | 0;

    // Enable ADC, interrupt, auto trigger, prescaler 128 (ADC clk ~125 kHz)
    ADCSRA = (1<<ADEN) | (1<<ADIE) | (1<<ADATE)
        | (1<<ADPS2) | (1<<ADPS1) | (1<<ADPS0);

    // Auto trigger source: Timer/Counter1 Compare Match B -> ADTS=101 (SFIOR)
    SFIOR &= ~((1<<ADTS2)|(1<<ADTS1)|(1<<ADTS0));
    SFIOR |= (1<<ADTS2) | (1<<ADTS0);

    // Start første konvertering (auto trigger tar resten)
    ADCSRA |= (1<<ADSC);
}

int main() {
    gpio_init();

```

```

timer1_init_262ms_trigger();
adc_init();

// ADC Noise Reduction Mode
set_sleep_mode(SLEEP_MODE_ADC);
sleep_enable();

sei();

// Tom event-loop: sov, våkn på ADC interrupt
for (;;) {
    sleep_cpu();
}
}

```

6 Resultat og observasjoner

Programmet bytter automatisk mellom Vin1 og Vin2, regner differansen og tenner riktig LED i henhold til intervallene. Når differansen er utenfor området, tennes alle LED-ene. Systemet tar ny måling hver ca. 262 ms, og main-loopen er tom slik oppgaven krever.

7 Konklusjon

Løsningen oppfyller kravene i Assignment 4 ved å bruke ADC Noise Reduction Mode, ADC auto trigger med ca. 262 ms intervall, tom event-loop og switch-baserte valg for LED-områder. Kondensatoren på AREF forbedrer stabiliteten i ADC-målingene, og LED-indikasjonen gir en enkel og tydelig visualisering av $V_{in2} - V_{in1}$.